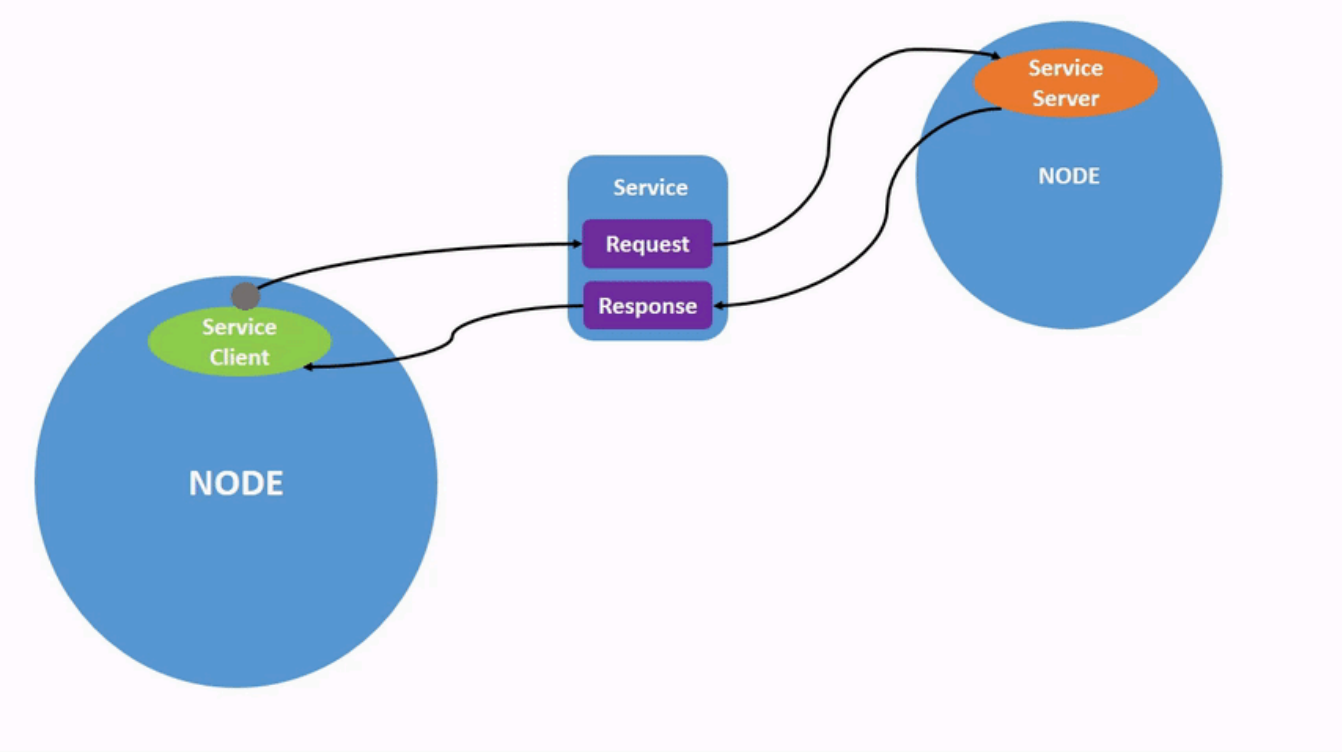
8. ROS2 Service Communication

1. Introduction to Service Communication

Service communication is a communication model based on request/response. In the communication between the two parties, the client sends the request data to the server, and the server responds with the result to the client.

The client/server model is as follows:



From the perspective of the service implementation mechanism, this you-ask-I-answer form is called the client/server model, abbreviated as the CS model. When the client needs certain data, it sends a request message for a specific service. After the server receives the request, it processes it and feeds back the response message.

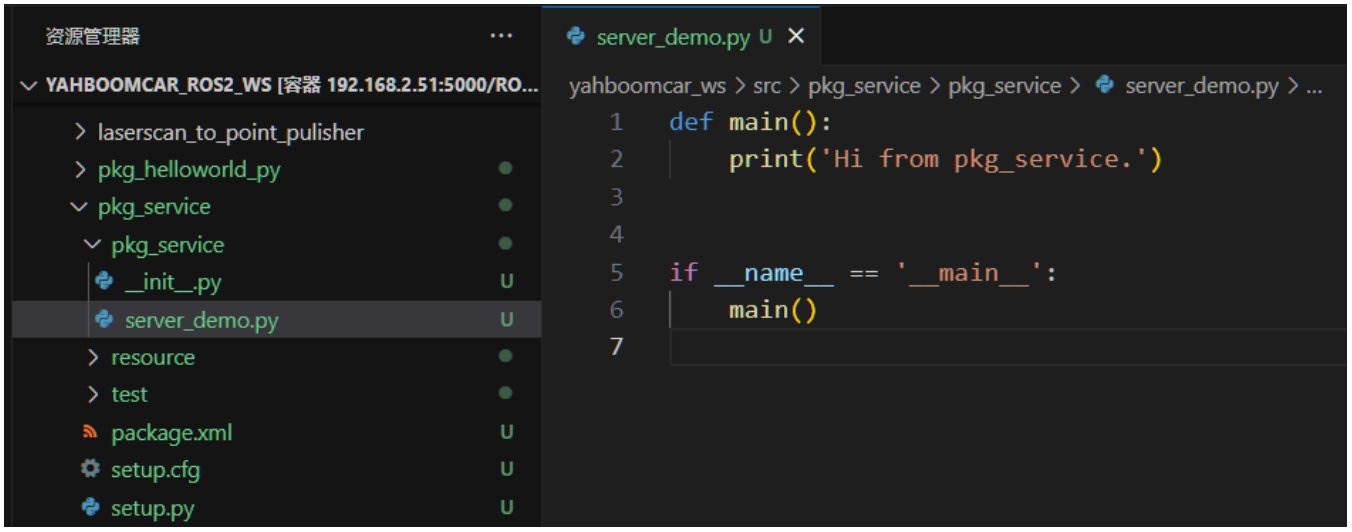
This communication mechanism is also very common in life. For example, the various web pages we often browse: at that moment your computer browser is the client, sending requests to the website server through a domain name or various operations, and the server returns the page data to be displayed after receiving the request.

2. Creating a New Function Package

- In the src directory of the workspace:

```
ros2 pkg create pkg_service --build-type ament_python --dependencies rclpy --node-name
server_demo
```

After executing the command above, the pkg_service function package is created, a server_demo node is created at the same time, and the relevant configuration files have already been set up.



3. Server Implementation

3.1 Create the Server

Next, edit [server_demo.py] to implement the server function and add the following code:

```
#Import the relevant library files
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class Service_Server(Node):
    def __init__(self,name):
        super().__init__(name)
        # Create a server using the create_service function; the parameters passed in are
respectively:
        # the service data type, the service name, the service callback function (that is,
the content of the service)
        self.srv = self.create_service(AddTwoInts, '/add_two_ints', self.Add2Ints_callback)
    # The content of the service callback function here adds two integers together and
returns the result
    def Add2Ints_callback(self,request,response):
        response.sum = request.a + request.b
        print("response.sum = ",response.sum)
        return response
def main():
    rclpy.init()
    server_demo = Service_Server("publisher_node")
    rclpy.spin(server_demo)
    server_demo.destroy_node()                  # Destroy the node object
    rclpy.shutdown()                            # Shut down the ROS2 Python interface
```

Focus on the service callback function, Add2Ints_callback. Besides self, the parameters that need to be passed in here are request and response. request is the parameter required by the service, and response is the feedback result of the service. request.a and request.b are the contents of the request part, and response.sum is the content of the response part. First let's look at what the data of the AddTwoInts type looks like.

- You can use the following command to view it

```
ros2 interface show example_interfaces/srv/AddTwoInts
```

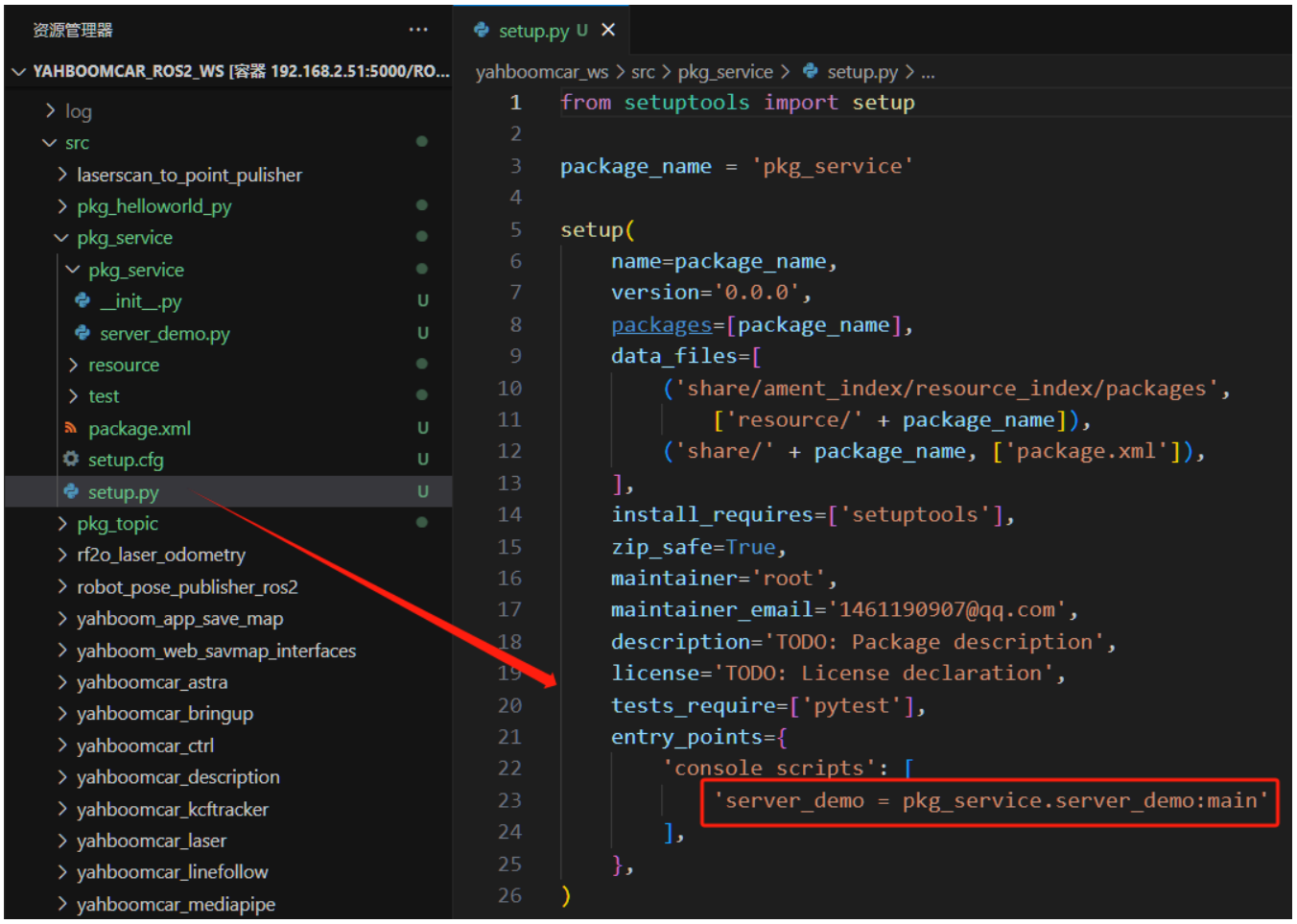
```
root@unbutu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 interface show example_interfaces/srv/AddTwoInts
int64 a
int64 b
---
int64 sum
```

"---" divides the data of this type into two parts, the upper part represents the request and the lower part represents the response. Then each of the fields has its own variable, such as int64 a and int64 b, so when passing parameters, you need to specify what the values of a and b are. Similarly, the feedback result also needs to specify what the value of sum is.

3.2 Edit the Configuration File

- Open setup.py and add the following to the console_scripts list

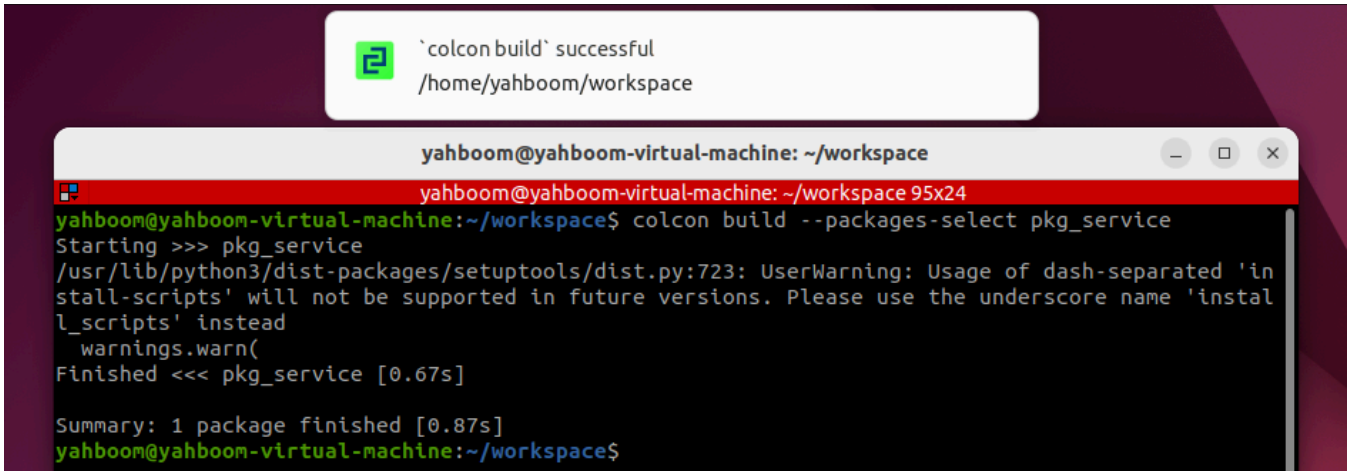
```
'server_demo = pkg_service.server_demo:main',
```



3.3 Build the Function Package

- Build the function package

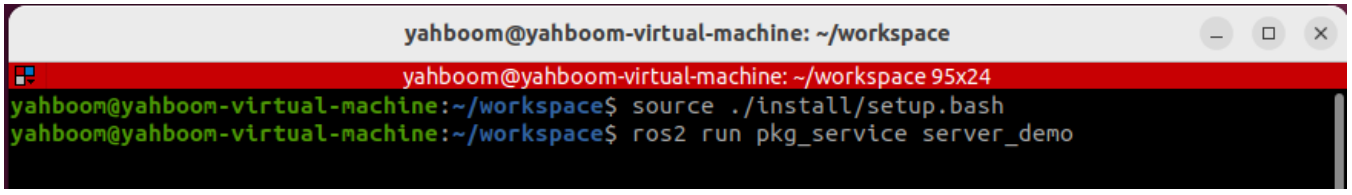
```
colcon build --packages-select pkg_service
```



3.4 Run the Program

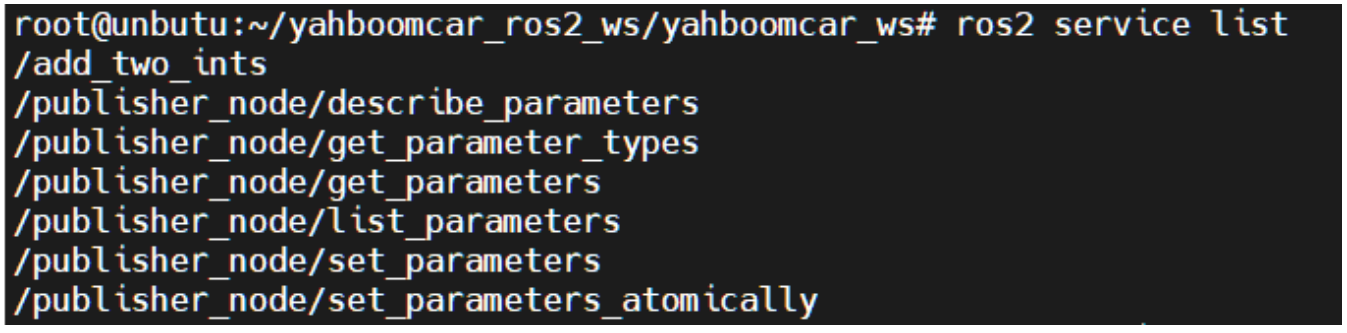
- Refresh the environment variables and then run the node

```
ros2 run pkg_service server_demo
```



After running, since this service is not called, there is no feedback data. We can call the service through the command line. First, query which services currently exist, open another terminal and enter:

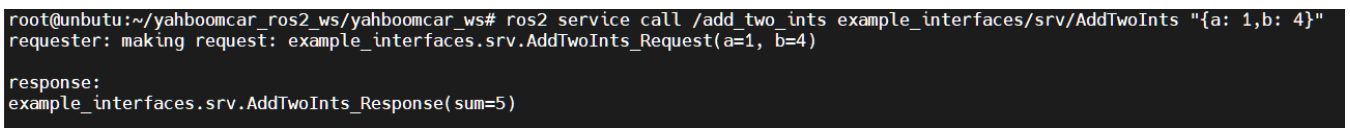
```
ros2 service list
```



/add_two_ints is the service we need to call. Call it with the following command, enter in the terminal:

```
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 1,b: 4}"
```

Here we assign the value of a to 1 and the value of b to 4, that is, we call the service to calculate the sum of 1 and 4:

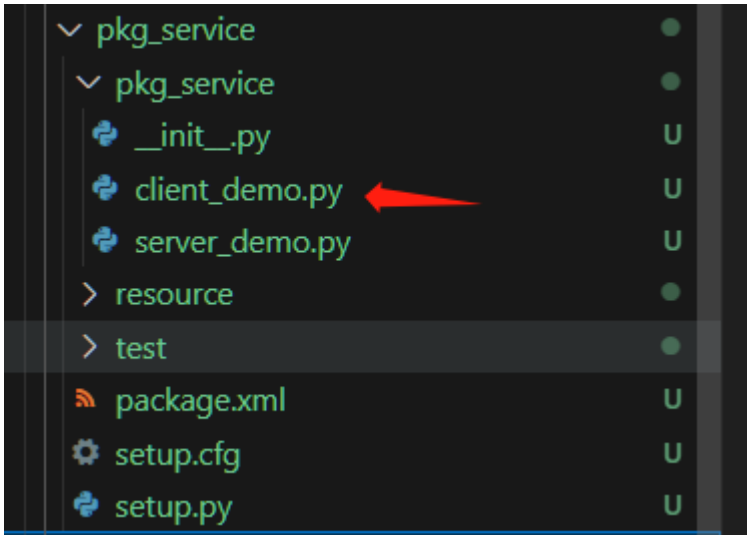


As can be seen from the figure above, after the service is called, the feedback result is 5, and the terminal running the server also prints the fed-back value.

4. Client Implementation

4.1 Create the Client

Create a new file [client_demo.py] in the same directory as [server_demo.py]



Next, edit [client_demo.py] to implement the client function and add the following code:

```
#Import the relevant libraries
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class Service_Client(Node):
    def __init__(self,name):
        super().__init__(name)
        # Create the client using the create_client function; the parameters passed in are
the service data type and the service topic name
        self.client = self.create_client(AddTwoInts,'/add_two_ints')
        # Loop and wait for the server to start successfully
        while not self.client.wait_for_service(timeout_sec=1.0):
            print("service not available, waiting again...")
        # Create the service request data object
        self.request = AddTwoInts.Request()

    def send_request(self):
        self.request.a = 10
        self.request.b = 90
        # Send the service request
        self.future = self.client.call_async(self.request)

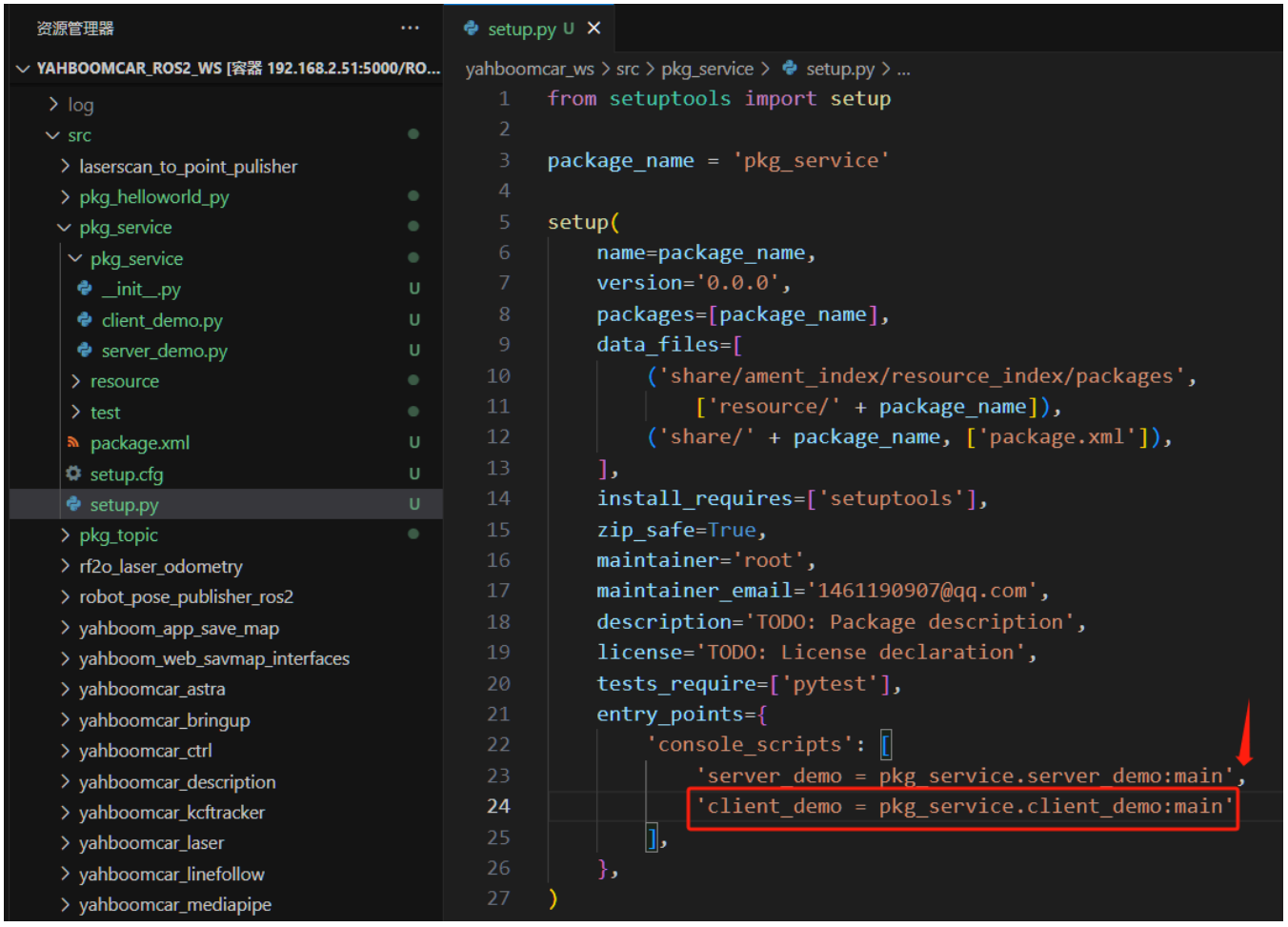
def main():
    rclpy.init() # Node initialization
    service_client = Service_Client("client_node") # Create the object
    service_client.send_request() # Send the service request
```

```
    while rclpy.ok():
        rclpy.spin_once(service_client)
        # Determine whether the data has been processed
        if service_client.future.done():
            try:
                # Get the feedback information of the service and print it
                response = service_client.future.result()
                print("service_client.request.a = ",service_client.request.a)
                print("service_client.request.b = ",service_client.request.b)
                print("Result = ",response.sum)
            except Exception as e:
                service_client.get_logger().info('Service call failed %r' % (e,))
        break
    service_client.destroy_node()
    rclpy.shutdown()
```

4.2 Edit the Configuration File

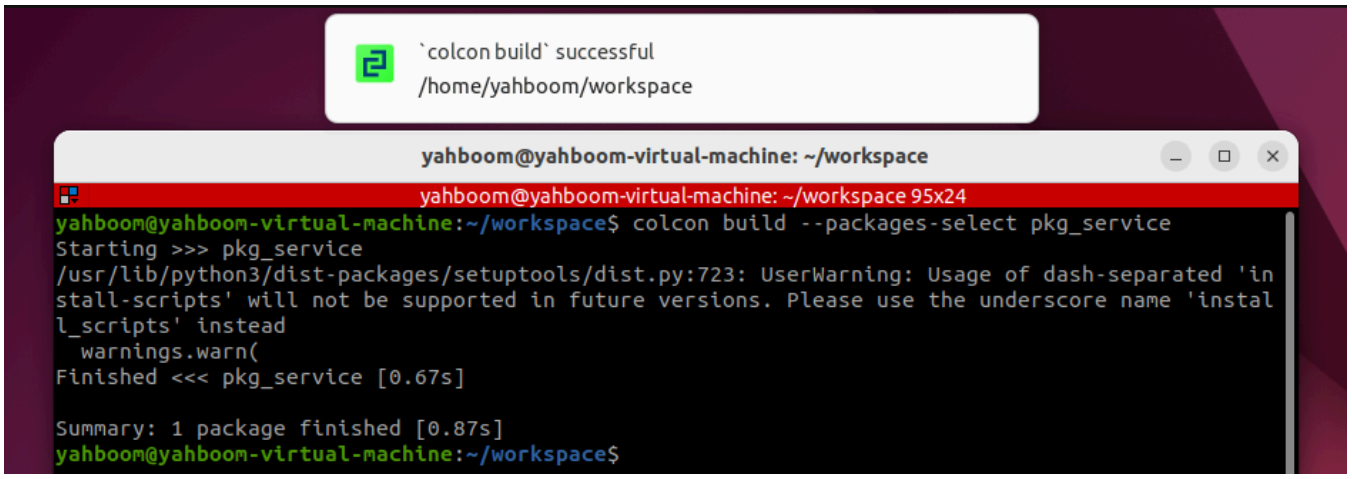
- Open setup.py and add the following to the console_scripts list

```
'client_demo = pkg_service.client_demo:main'
```



4.3 Build the Function Package

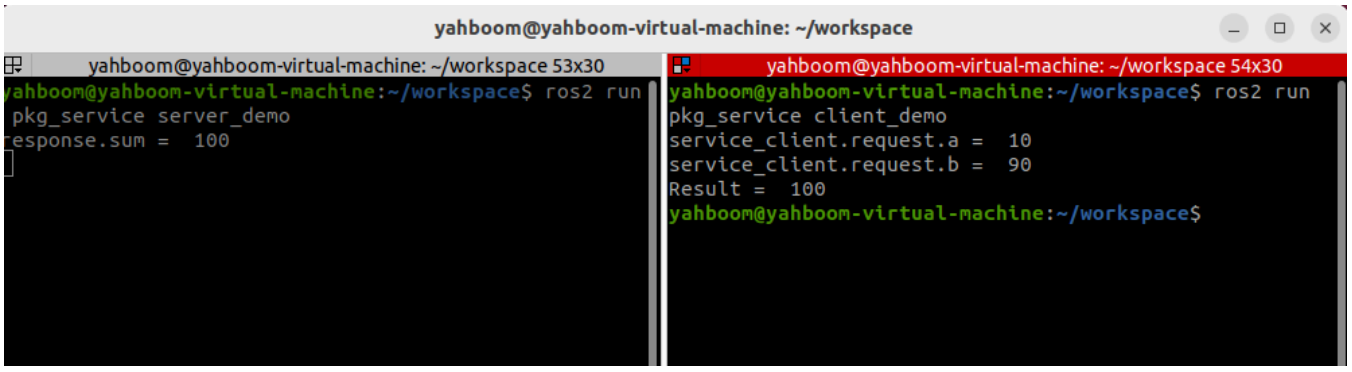
```
colcon build --packages-select pkg_service
```



4.4 Run the Program

- Refresh the environment variables and then run the node

```
#Start the server node
ros2 run pkg_service server_demo
#Start the client node
ros2 run pkg_service client_demo
```



First run the server, then run the client. The client provides a=10, b=90, and the server performs the sum, obtaining the result 100, which is printed in the terminals of both.